VERIQTA

The DevOps Side Project Playbook

10 IDEAS That Get You Hired



REAL PROJECTS | PRACTICAL SKILLS | PRODUCTION PRACTICES | INTERVIEW READY | PORTFOLIO READY

@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

Page 1

VERIQTA

Project 01: Build a Deployment That Rejects Dangerous Releases

Scenario

A team deploys successfully, but its latest release increases API errors and breaks customer transactions. The CI/CD pipeline still reports success.

Engineering Insight

Passing unit tests does not establish that a release is safe for production.

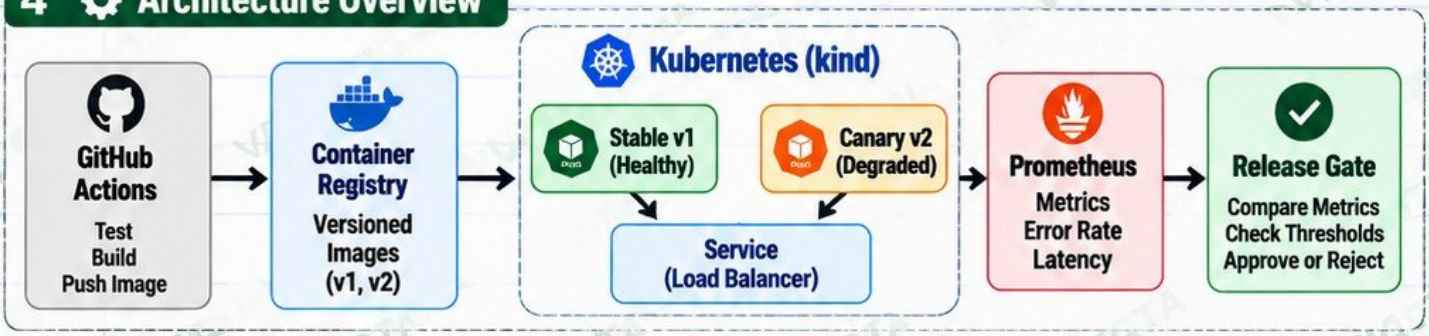
2 Goal

Build a release pipeline that automatically evaluates production health before approving a deployment.

3 What You Will Build

- A small API with two versions (healthy and degraded)
- GitHub Actions pipeline to test, package and publish images
- Deploy to a local Kubernetes cluster using kind
- Prometheus to collect request error rates and latency
- Canary deployment with automated health comparison
- Release gate to reject unhealthy canary and keep stable version

4 Architecture Overview



5 </> Sample API Versions

Healthy Version (v1)

```python
from flask import Flask
app = Flask(__name__)

@app.route("/api")
def api():
    return {"status": "ok",
            "version": "v1"}, 200
```

Degraded Version (v2)

```python
from flask import Flask
import random
app = Flask(__name__)

@app.route("/api")
def api():
    if random.random() < 0.2:
        return {"error": "failure"}, 500
    return {"status": "ok",
            "version":"v2"}, 200
```

6 Canary Deployment (Kubernetes)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-canary
spec:
  replicas: 1
  selector:
    matchLabels:
      app: api
      version: v2
  template:
    metadata:
      labels:
        app: api
        version: v2
```

Prometheus Query (Error Rate)

```
sum(rate(http_requests_total{version="v2",code=~"5.."}[5m]))
/
sum(rate(http_requests_total{version="v2"}[5m]))
```

8 Release Gate Logic

- Check canary error rate < 2%
- Check latency (p95) < 300ms
- If thresholds fail → reject and keep stable version
- If healthy → promote to full traffic

Deliverable

A working release pipeline with health-based promotion and rollback evidence.

@veriqta Instagram
@veriqta GitHub
@veriqta LinkedIn
@veriqta Telegram
@veriqta X
@veriqta Discord

VERIQTA

Project 01: Prove Your Pipeline Can Stop a Bad Deployment

1 Failure Challenge

Introduce an application version that returns HTTP 500 errors for 20% of requests.

2 Investigate

- Does the canary exceed the allowed error rate?
- Can the pipeline identify the failing release?
- Does the release stop before receiving full traffic?
- Does the system preserve the healthy application version?

4 Pipeline Output (GitHub Actions)

```
>> Run: Check canary health
Canary error rate: 21.4%
Threshold: 2%
❌ Canary failed health check
🟨 Stopping promotion...
✅ Keeping stable version (v1) live
Deployment aborted.
```

5 Kubernetes Services

```
$ kubectl get pods -l app=api
NAME                  READY   STATUS    VERSION
api-v1-679c8f8d97     2/2     Running   v1 (stable)
api-v2-5f9d7c6cbf     2/2     Running   v2 (canary)
```

```
$ kubectl get svc api
NAME   TYPE           CLUSTER-IP      PORT(S)
api    LoadBalancer   10.96.123.45    80:31234/TCP
```

7 GitHub Evidence (Repository)

- .github/workflows/deploy.yml (GitHub Actions workflow)
- Kubernetes manifests (deployment, service, canary config)
- Prometheus queries and dashboards (Grafana screenshots)
- Pipeline logs showing rejection
- Short video or screenshots of the failure test and recovery
- README with setup, results, and key learnings

app.py (v2 - degraded)

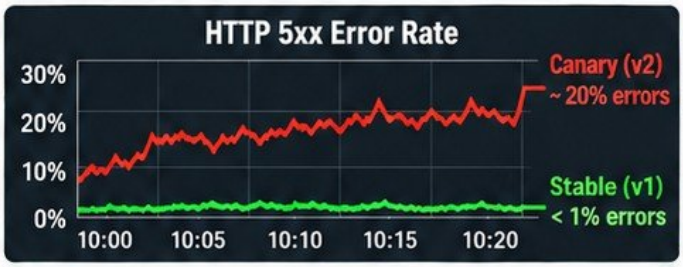
```python
from flask import Flask
import random
app = Flask(__name__)

@app.route("/api")
def api():
    if random.random() < 0.2:
        return {"status": "error"}, 500
    return {"status": "ok", "version": "v2"}, 200
```

3 Prometheus: Compare Error Rates

```
# Error rate for canary (v2)
sum(rate(http_requests_total{version="v2",
    code="5.."}[5m]))

# Error rate for stable (v1)`
sum(rate(http_requests_total{version="v1",
    code="5.."}[5m]))
```



6 Verification Results

Test	Expected	Actual	Result
Deploy healthy v1	< 1% errors	0.8%	✅
Deploy canary v2 (degraded)	> 2% errors	21.4%	✅
Pipeline blocks promotion	Stop canary	Stopped	✅
Stable version remains live	v1 still serving	v1 healthy	✅
Application recovers	Normal error rate	0.7%	✅

8 Interview Defense

Q: Why is a green CI/CD pipeline insufficient evidence that a deployment is safe?

A: A pipeline usually validates code, tests, and build artifacts. It cannot fully validate real user impact, runtime behavior, external dependencies, performance, or error rates in production. Only production health signals can confirm a release is safe.

VERIQTA

Project 02: Build a Cloud Infrastructure Drift Detective

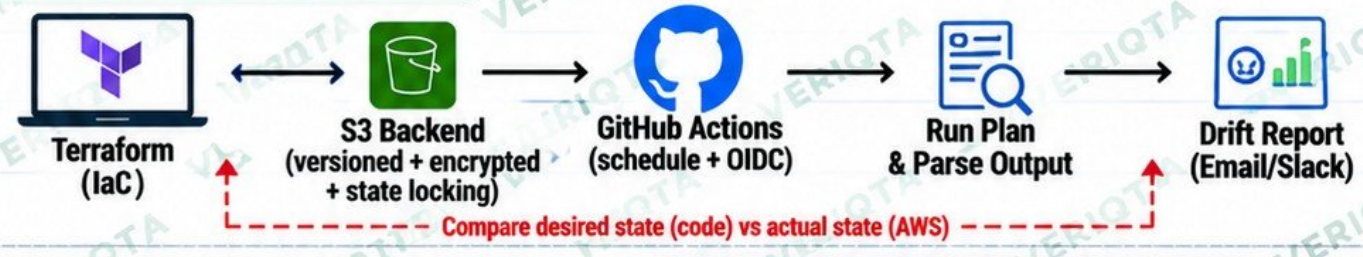
1 Scenario

An engineer manually changes a cloud security group. Terraform code remains unchanged, but the deployed environment no longer matches the approved architecture.

2 Goal

Build a system that detects, classifies, and reports infrastructure drift.

3 Architecture Overview



4 Terraform Backend (S3)

```
terraform {
  backend "s3" {
    bucket         = "veriqta-terraform-state"
    key            = "drift-demo/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "veriqta-terraform-lock"
  }
}
```

5 GitHub Actions Workflow (OIDC)

```
name: Terraform Drift Detection
on:
  schedule:
    - cron: '0 2 * * *'  # daily
  workflow_dispatch:
permissions:
  id-token: write
  contents: read
jobs:
  drift-detect:
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/TerraformDrift
          aws-region: us-east-1
      - name: Terraform Plan
        run: terraform plan -detailed-exitcode -no-color > plan.txt
```

6 Run and Detect Drift

```
# Initialize and run plan
terraform init
terraform plan -detailed-exitcode -no-color
# Exit codes
0 = No changes (in sync)
1 = Error
2 = Changes detected (drift)
```

7 Example Drift Output

```
# aws_security_group.app_sg will be updated
~ resource "aws_security_group" "app_sg" {
    name   = "app-sg"
    ingress {
  -   cidr_blocks = ["0.0.0.0/0"]   # manually added
  +   cidr_blocks = ["10.0.0.0/16"] # expected
    }
}
```

8 Classify and Report

Resource	Change	Classification	Security Impact
aws_security_group app_sg	Ingress 0.0.0.0/0 added	Unexpected	High (public access)
aws_instance web	Instance type changed	Expected	Low (approved change)
aws_s3_bucket logs	Versioning enabled	Routine	None

9 Engineering Insight

Infrastructure as Code does not prevent manual changes. It makes desired state explicit and drift detectable.

10 Deliverable

An automated drift detection system with actionable reports.

VERIQTA

Project 02: Detect a Dangerous Change Without Destroying Infrastructure

1 Failure Challenge

Manually modify a security group to permit unintended public access (e.g., open port 22 or 0.0.0.0/0).

2 Investigate

- Which resource changed?
- Does the change introduce public exposure?
- Can the workflow detect the difference without applying changes?
- How should the team decide to revert or formally adopt the change?

3 Example: Manual Change to Security Group

Original (Terraform)

Security Group (app-sg)

Type	Port	Source
SSH	22	10.0.0.0/16
HTTP	80	0.0.0.0/0

→

Manual Change (in Console)

Security Group (app-sg)

Type	Port	Source
SSH	22	0.0.0.0/0 ← Dangerous
HTTP	80	0.0.0.0/0

→

Drift Detection (terraform plan)

Security Group (app-sg)

Type	Port	Source
SSH	22	10.0.0.0/16 ← Expected
SSH	22	0.0.0.0/0 ← Detected Drift

4 Terraform Plan Output (Drift Detected)

```
# aws_security_group.app-sg will be updated
~ resource "aws_security_group" "app_sg" {
    id      = "sg-0abc123"
  ~ ingress {
    - from_port   = 22
      to_port     = 22
    ~ cidr_blocks = ["0.0.0.0/0"]  # added manually
    - cidr_blocks = ["10.0.0.0/16"] # in code
}

Plan: 0 to add, 1 to change, 0 to destroy.
```

5 GitHub Actions Detection Workflow

```yaml
name: Terraform Drift Detection
on:
  schedule:
    - cron: '0 2 * * *'  # daily
  workflow_dispatch:
permissions:
  id-token: write
  contents: read
jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/TerraformDrift
          aws-region: us-east-1
      - name: Terraform Plan
        run: terraform plan -detailed-exitcode -no-color > plan.txt
```

6 Drift Report (Sample)

Resource	Change	Risk Level	Security Impact
aws_security_group app-sg	Ingress 22 0.0.0.0/0 added	High	Public SSH access from anywhere
aws_instance web	No change	None	-
aws_s3_bucket logs	No change	None	-

7 Recommended Actions

- Revert the manual change to match Terraform code
- Investigate who made the change (CloudTrail)
- Decide if the change is required (temporary or permanent)
- If required, update Terraform code and document the reason
- Run terraform apply only after review and approval

8 Verification

- Before and after drift report (screenshots)
- Terraform plan output showing detected change
- Workflow detects drift without applying changes
- Remediation restores secure configuration

9 GitHub Evidence

- Terraform modules (main.tf, variables.tf)
- GitHub Actions workflow file
- Sample sanitized plan output
- Risk classification report (Markdown or CSV)
- Recovery documentation (step by step)

10 Interview Defense

Q: Would you automatically run terraform apply whenever drift is detected? Why or why not?

A: No. Drift may be intentional, temporary, or caused by provider behavior. Always review, classify the change, assess security impact, and get approval before applying.

VERIQTA

Project 03: Build a Kubernetes Failure Investigation Lab

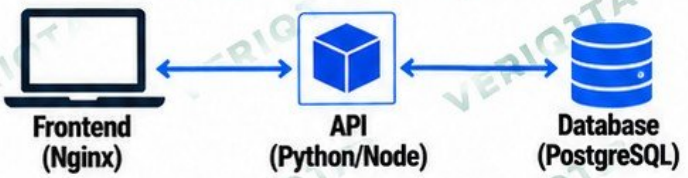
1 Scenario

An application experiences intermittent downtime, but its Pods appear to be running.

2 Goal

- Build a Kubernetes troubleshooting environment that deliberately reproduces common production failures and teaches you how to investigate them.

3 Application Architecture



4 What You Will Build

- Deploy a multi-service application (frontend, API, database).
- Configure resource requests, limits, readiness and liveness probes.
- Introduce DNS failures, missing ConfigMaps, and resource pressure.
- Collect Kubernetes events, container logs, and resource metrics.
- Build diagnostic scripts to capture evidence without modifying the cluster.
- Document how to distinguish application faults from cluster faults.

5 Kubernetes Manifests (Example)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: api
        image: veriqta/api:v1
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "512Mi"
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 20
          periodSeconds: 10
```

6 Failure Scenarios

DNS Failure
Break DNS resolution for the API service.

Missing ConfigMap
Remove or rename a required ConfigMap.

Resource Pressure
Create a load that exhausts CPU or memory.

Unhealthy Application
Make the API return HTTP 500 errors for some requests.

7 Useful Investigation Commands

```
# Check pods and status
kubectl get pods -o wide

# Inspect events
kubectl get events --sort-by=.lastTimestamp

# Describe a pod
kubectl describe pod <pod-name>

# View logs (previous container)
kubectl logs <pod-name> --previous

# Check endpoints
kubectl get endpoints <service-name>

# Check DNS resolution
kubectl exec -it <pod-name> -- nslookup api

# View resource usage
kubectl top pods
kubectl top nodes
```

8 Collect Evidence (Scripts)

```
#!/bin/bash
# gather-evidence.sh
NAMESPACE=$1
mkdir -p evidence/$NAMESPACE
kubectl get pods -n $NAMESPACE -o wide > evidenc
kubectl get events -n $NAMESPACE --sort-by=.last
kubectl get svc,ep -n $NAMESPACE -o wide > evide
kubectl logs -n $NAMESPACE --all-containers > ev
kubectl top pods -n $NAMESPACE > evidence/metric
```

9 Engineering Insight

A running Pod is not proof that an application is serving requests successfully.

10 Deliverable

- A reusable Kubernetes incident investigation repository.
- Manifests, scripts, failure scenarios, and troubleshootion guides.
- Clear documentation on application vs cluster faults.

VERIQTA

Project 03: Diagnose Five Failures Using Evidence

1 Failure Challenges

1. Pod enters CrashLoopBackOff.
2. Container terminates with OOMKilled.
3. Service has no ready endpoints.
4. Application cannot resolve a dependency through DNS.
5. Deployment becomes unavailable because readiness checks fail.

2 Investigation Commands

```
# Describe pod and check events
kubectl describe pod <pod-name>
# View current and previous logs
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
# Check endpoints and endpointslices
kubectl get endpointslices
kubectl get endpoints <service-name>
# Check resource usage
kubectl top pods
kubectl top nodes
# Check DNS resolution from a pod
kubectl exec -it <pod-name> -- nslookup <service>
kubectl exec -it <pod-name> -- cat /etc/resolv.conf
```

3 Pod CrashLoopBackOff



STATUS
CrashLoopBackOff

```
$ kubectl get pods
NAME      READY   STATUS
api-7d8   0/1     CrashLoopBackOff

$ kubectl describe pod api-7d8
Events:
  Type     Reason     Message
  ----     ------     -------
  Warning  BackOff    Back-off
                      restarting
                      failed container
```

Root Cause
Application error or misconfiguration.

Fix
Fix the error and redeploy.

4 Container OOMKilled



STATUS
OOMKilled

```
$ kubectl describe pod api-7d8
Last State:
   Terminated:
     Reason:     OOMKilled
     Exit Code:  137

$ kubectl logs api-7d8
(previous)
...
```

Root Cause
Container exceeded memory limit.

Fix
Increase memory limit or optimize the application.

5 Service No Endpoints



STATUS
No Ready Endpoints

```
$ kubectl get endpoints api
NAME   ENDPOINTS   AGE
api    <none>      2m

$ kubectl get endpointslices
NAME          ADDRESSES   READY
api-abc123    <none>      false
```

Root Cause
Pods not ready or selector mismatch.

Fix
Fix labels, selector or application health.

6 DNS Resolution Failure



STATUS
DNS Lookup Fails

```
$ kubectl exec -it api-7d8
-- nslookup db
Server:    10.96.0.10
Address:   10.96.0.10#53
** server can't find db:
   NXDOMAIN
```

Root Cause
Incorrect service name, missing service or DNS configuration issue.

Fix
Check service name, DNS configuration and CoreDNS status.

7 Readiness Check Failed



STATUS
Not Ready

```
$ kubectl get pods
NAME      READY   STATUS
api-7d8   0/1     Running

$ kubectl describe pod api-7d8
Readiness probe failed:
HTTP probe failed with
statuscode: 500
```

Root Cause
Application not ready or failing health check.

Fix
Fix application or adjust readiness probe configuration.

8 Verification

Failure	Symptom	Root Cause	Fix Applied	Recovered
CrashLoopBackOff	Pod restarting	App error	Fix config	✓
OOMKilled	Exit code 137	Memory limit	Increase limit	✓
No endpoints	<none>	Pods not ready	Fix selector	✓
DNS failure	NXDOMAIN	DNS/service	Fix DNS	✓
Readiness failed	HTTP 500	Health check	Fix app/probe	✓

9 GitHub Evidence

- Five incident reports (one per failure scenario)
- Diagnostic scripts (bash)
- Sanitized logs and outputs
- Kubernetes manifests
- Troubleshooting decision trees
- README with setup, results, and key learnings

10 Interview Defense

Q: A Kubernetes Service exists, but requests fail. What evidence would you inspect before changing anything?

A: I would check:

1. Service endpoints/endpointslices
2. Pod status and logs
3. Events and recent changes
4. DNS resolution
5. Application health and metrics

VERIQTA

Project 04: Build an Automated Cloud Cost Anomaly Investigator

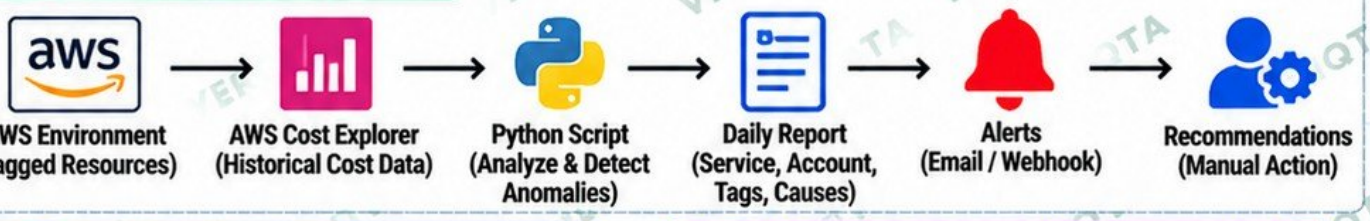
1 Scenario

A development team's cloud bill increases unexpectedly even though application traffic remains stable.

2 Goal

Build a system that identifies unusual infrastructure spending and explains possible causes.

3 Architecture Overview



4 Sample Python Script (Cost Analysis)

```python
import boto3
from datetime import datetime, timedelta

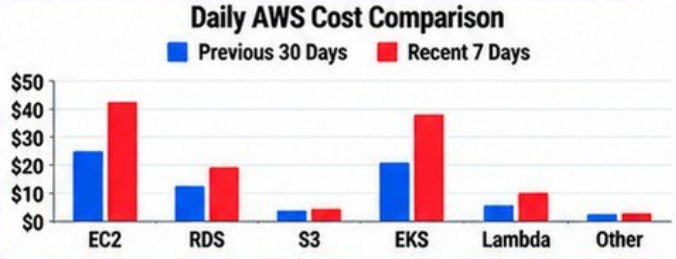
client = boto3.client('ce')

end = datetime.utcnow().date()
start = end - timedelta(days=30)

response = client.get_cost_and_usage(
    TimePeriod={'Start': str(start), 'End': str(end)},
    Granularity='DAILY',
    Metrics=['UnblendedCost'],
    GroupBy=[{'Type': 'DIMENSION', 'Key': 'SERVICE"}]
)

for item in response['ResultsByTime']:
    print(item['TimePeriod']['Start'], item['Total'])
```

5 Example Output (Cost Comparison)



Service	Previous 30 Days	Recent 7 Days	Change	Status
EC2	$48.40	$42.10	+128%	High
RDS	$12.20	$19.30	+58%	High
S3	$3.10	$3.50	+13%	Normal
EKS	$20.50	$38.60	+88%	High
Lambda	$5.40	$9.20	+70%	High
Other	$2.10	$2.30	+10%	Normal

6 Anomaly Detection Logic

```python
# Simple anomaly detection (example)
threshold = 0.5  # 50% increase

if recent_cost > (baseline_cost * (1 + threshold)):
    print(f"Anomaly detected: {service}")
    print(f"Increase: {((recent_cost-baseline_cost)
          / baseline_cost)*100:.1f}%")
```

7 Alert Example (Email (Webhook)

```
Subject: [VERIQTA] AWS Cost Anomaly Detected

  Service: EC2
  Increase: +128%
  Recent 7 Days: $42.10
  Previous 30 Days: $18.40
  Possible Causes:
   - Unused instances
   - Auto Scaling misconfiguration
   - Increased instance size
   - Unintended workloads
Please investigate: https://console.aws.amazon.com/cost/
```

8 Recommended Actions

- Investigate resources without proper tags
- Check for unused or orphaned resources
- Review Auto Scaling and instance sizes
- Verify recent deployments or configuration changes
- Consider rightsizing and reserved instances
- Document findings before making changes

9 Engineering Insight

Cost increases can indicate forgotten infrastructure, excessive retries, inefficient scaling, or unexpected workloads.

10 Deliverable

- A cloud cost investigation tool with explainable alerts.
- Python scripts and documentation.
- Example reports and alert samples.
- Recommendations without automatic resource deletion.

VERIQTA

Project 04: Investigate a 40% Cloud Cost Increase

1 Failure Challenge

Use synthetic cost data representing an unexpected 40% spending increase.

2 Investigate

- Which services contributed most?
- Did usage increase, or did the unit cost change?
- Are resources missing ownership tags?
- Does the anomaly correspond with a deployment or infrastructure change?
- Could a delayed billing update distort the analysis?

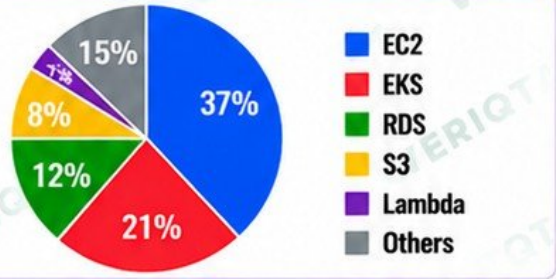
3 Cost Comparison (Example)



4 Top Services by Cost Increase

Service	Previous Cost	Current Cost	Increase	% Change
EC2	$420	$735	+$315	+75%
RDS	$280	$420	+$140	+50%
S3	$150	$240	+$90	+60%
EKS	$310	$430	+$120	+39%
Lambda	$120	$150	+$30	+25%
Others	$140	$213	+573	+52%

5 Cost Breakdown (Current Month)



6 Example: Identify Missing Tags

Resource	Service	Monthly Cost	Owner Tag	Status
i-0abc123	EC2	$210	-	Missing
rds-db-01	RDS	$140	dev-team	Tagged
s3-logs	S3	$90	-	Missing
eks-node-1	EC2	$160	-	Missing
lambda-func	Lambda	$30	data-team	Tagged

7 Python Analysis Script (Example)

```python
import boto3
import pandas as pd
ce = boto3.client('ce')
response = ce.get_cost_and_usage(
    TimePeriod={'Start': start_date, 'End': end_date},
    Granularity='DAILY',
    Metrics=['UnblendedCost'],
    GroupBy=[{'Type': 'DIMENSION', 'Key': 'SERVICE'}]
)
df = pd.DataFrame(response['ResultsByTime'])
# compare with historical baseline and detect > 40% increase
threshold = 0.4
```

8 Example Alert Output (Email/Webhook)

Subject: [VERIQTA] AWS Cost Anomaly Detected
Total Increase: +40% ($568)
Top Contributors:
- EC2: +$315 (+75%)
- RDS: +$140 (+50%)
- S3 : +$90 (+60%)
- EKS: +$120 (+39%)

Possible Causes:
- Unused instances
- Auto scaling misconfiguration
- Missing resource tags
- Increased storage or logs

9 Verification

- Ranked cost report with top contributors
- Evidence of usage vs unit cost changes
- Identification of missing tags
- Correlation with deployments/changes
- Estimated savings recommendations

10 GitHub Evidence

- Python scripts (cost analysis)
- Synthetic cost dataset (CSV/JSON)
- Sample reports and charts
- Alert templates (email/webhook)
- Investigation documentation (step by step)

11 Interview Defense

Q: How would you distinguish legitimate business growth from infrastructure waste?
A: Compare usage metrics, application traffic, deployments, resource tags, and business context. Investigate anomalies before making changes.

VERIQTA

Project 05: Build a Secrets Leak Prevention Pipeline

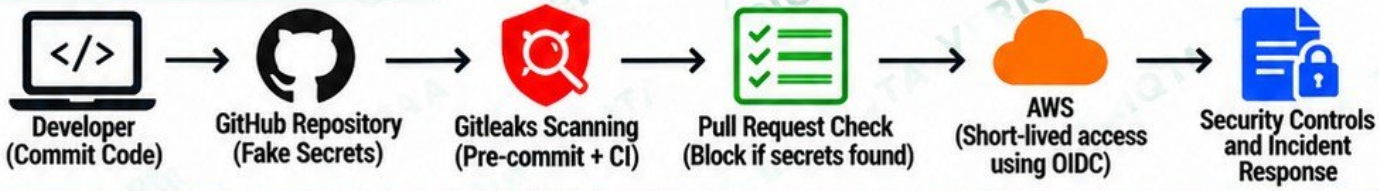
1 Scenario

A developer accidentally commits a cloud access key into a repository.

2 Goal

Build a CI/CD security control that detects credential exposure before code reaches production.

3 Solution Architecture



4 Fake Secret Example (for Testing)

```
# fake-secret-example.py

AWS_ACCESS_KEY_ID = "AKIAFAKEEXAMPLE1234"
AWS_SECRET_ACCESS_KEY = "fakeSecretKey123/
x8QwJ9Z"

# Add other examples
API_KEY = "sk-fake-1234567890abcdef"
DATABASE_URL = "postgres://user:password@
db.example.com:5432/app"
```

5 GitHub Actions Workflow (Gitleaks)

```
name: Secret Scan
on:
  pull_request:
  push:
    branches: [ main ]
jobs:
  gitleaks:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Run Gitleaks
        uses: gitleaks/gitleaks-action@v2
        with:
          config-path: .gitleaks.toml
          fail: true
```

6 Local Pre-commit Scanning

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/gitleaks/gitleaks
    rev: v8.18.0
    hooks:
      - id: gitleaks
        args: ["--config", ".gitleaks.toml"]
```

7 Required Pull Request Check

✗	Gitleaks / gitleaks	Failed	12s
✓	CI / build	Successful	45s
✓	Unit Tests	Successful	1m 12s

This branch has required checks that must pass before merging.

8 Short-Lived AWS Authentication (OIDC)

```
- name: Configure AWS credentials (OIDC)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789012:role/GitHubActions
    aws-region: us-east-1
```

9 Credential Incident Response Checklist

- Revoke the exposed credentials immediately
- Identify where the secret was exposed
- Review access logs for unauthorized activity
- Rotate keys and update applications
- Remove the secret from Git history
- Add detection and prevention controls
- Document the incident and lessons learned

10 Engineering Insight

Removing a secret from the latest commit does not prove it was never exposed.

11 Deliverable

- A reusable security pipeline with tested detection.
- Pre-commit and CI/CD scanning.
- Short-lived cloud authentication using OIDC.
- A documented credential incident response procedure.

VERIQTA

Project 05: Simulate a Credential Exposure Incident

1 Failure Challenge

Commit a synthetic credential and demonstrate that the pipeline blocks the pull request.

2 Investigation Questions

- Where was the credential exposed?
- Did the secret reach Git history or CI logs?
- Could an attacker have accessed it?
- What would need revocation or rotation in a real incident?
- How would you confirm the replacement credential has least-privilege access?

3 Simulate the Exposure (Commit Fake Secret)

```
# app/config.py

AWS_ACCESS_KEY_ID = "AKIAFAKE1234567890"
AWS_SECRET_ACCESS_KEY = "fakeSecretKey123/
x8QwJ9Z"

# Commit and push
git add app/config.py
git commit -m "Add config (testing)"
git push origin feature/test-secret
```

✕ Fake credential committed

4 Pipeline Blocks the Pull Request

✕ **Gitleaks / Secret Scan** — Failed

1 error detected

▾ Detected Secrets (1)

AWS Access Key ID	app/config.py:3
AKIAFAKE1234567890	

✕ Checks have failed
1 failing check

Merge pull request

Pull request is blocked until the secret is removed and the issue is resolved.

5 Check Git History and CI Logs

▸ Check Git History

```
git log -p --all -S "AKIAFAKE"
git rev-list --all | xargs -I {}
git show {} | grep -n "AKIAFAKE"
```

```
commit a1b2c3d4
Author: developer
Date:   Oct 25 2026

   +AWS_ACCESS_KEY_ID =
    "AKIAFAKE1234567890"
```

▸ Check CI Logs

```
gh run view <run-id> --log
```

```
INFO   gitleaks scan started
INFO   scanning files...
WARN   detected: AWS Access Key ID
       FILE: app/config.py:3
       SECRET: AKIAFAKE1234567890
       RULE: aws-access-key
ERROR  secret detected - failing job
```

6 Incident Response Steps (Real Scenario)

1. Revoke the exposed credentials immediately.
2. Identify where the secret was exposed (Git, logs, artifacts, etc).
3. Rotate keys and update applications.
4. Review access logs for unauthorized activity.
5. Implement detection and prevention controls.
6. Document the incident and lessons learned.

7 Verification

- Pipeline detects the secret and blocks the pull request.
- Secret is removed from the code and Git history.
- CI logs and artifacts are checked for exposure.
- Credentials are rotated (simulated) and verified.
- Documentation of the incident and recovery is completed.

8 GitHub Evidence

- GitHub Actions security workflow (Gitleaks).
- Repository with fake credential tests.
- Sample failed workflow logs (redacted).
- Remediation commit and rotation steps.
- Incident runbook and documentation.

9 Engineering Insight

Removing a secret from the latest commit does not prove it was never exposed.

10 Interview Defense

Q: Why is deleting a leaked secret from Git insufficient incident containment?

A:
1. The secret may still exist in **Git history**, forks, or cached clones.
2. It may be exposed in CI logs, artifacts, or external systems.
3. Attackers could have already accessed and used it.
4. The correct response is to revoke, rotate, investigate, and monitor.

VERIQTA

Project 06: Build an SLO-Based Production Alerting System

1 Scenario

Engineers receive hundreds of CPU alerts, yet the team misses an outage affecting real customers.

2 Goal

- Build a monitoring system that prioritizes
- customer-visible reliability over noisy infrastructure thresholds.

3 System Architecture



4 Instrument a Sample API (Python)

```python
from flask import Flask, request
from prometheus_client import Counter, Histogram,
    generate_latest

app = Flask(__name__)

REQUEST_COUNT = Counter('http_requests_total',
 'Total HTTP requests', ['method', 'endpoint', 'status'])
REQUEST_LATENCY = Histogram('http_request_duration_seconds',
 'Request latency')

@app.route('/api')
def api():
    REQUEST_COUNT.labels(method='GET', endpoint="/api',
    status='200').inc()
    with REQUEST_LATENCY.time():
        return {'message': 'hello'}, 200
@app.route('/metrics')
def metrics():
    return generate_latest(), 200
```

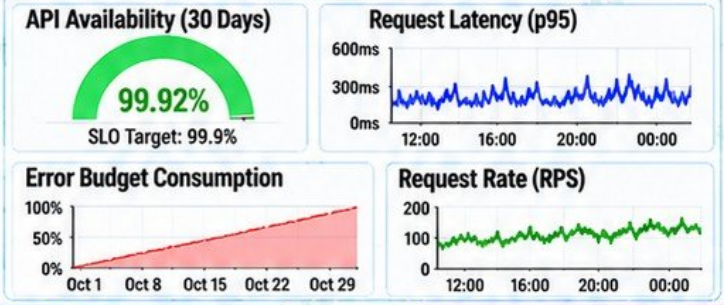
5 Define SLI, SLO and Error Budget

Component	Definition
SLI (Availability)	Percentage of successful requests (HTTP 2xx/3xx).
SLI (Latency)	Percentage of requests under target latency (e.g. < 300ms).
SLO	99.9% availability over 30 days (≤ 0.1% errors).
Error Budget	0.1% of total requests can fail.

6 Alert Rules (Multi-Window Burn Rate)

```yaml
- alert: HighErrorBudgetBurn
  expr: |
  (
    rate(http_requests_total{status=~"5.."}[5m])
    /
    rate(http_requests_total[5m]) > 0.14
  ) and
  (
    rate(http_requests_total{status=~"5.."}[1h])
    /
    rate(http_requests_total[1h]) > 0.02
  )
  for: 5m
  labels:
   severity: page
  annotations:
    summary: "High error budget burn detected"
    description: "Error rate is {{ $value | humanizePercentage }}
  and burning the 30d error budget too quickly."
  runbook: "https://github.com/veriqta/runbooks/blob/main/api.md"
```

7 Grafana Dashboard (Example)



8 Verification

- API metrics visible in Prometheus.
- Dashboards display availability, latency and error budget.
- Alerts trigger only on real reliability issues.
- Notifications include runbook links.
- Test an outage and confirm alert behavior.

9 Engineering Insight

High CPU utilization is not automatically a customer incident. Rapid error-budget consumption may be.

10 Deliverable

- Instrumented sample API.
- Prometheus, Alertmanager and Grafana setup.
- SLI, SLO and alert rules.
- Dashboards and screenshots.
- Runbook and documentation.

11 Interview Defense

Q: Why is high CPU utilization not automatically a customer incident?

A: Because high CPU may not impact user requests. Customer impact depends on SLI/SLO metrics such as availability and latency, not just infrastructure utilization.

VERIQTA

Project 06: Trigger an Outage and Measure the Alerting Quality

1 Failure Challenge

Introduce a controlled period of elevated HTTP 500 responses.

2 Investigation Questions

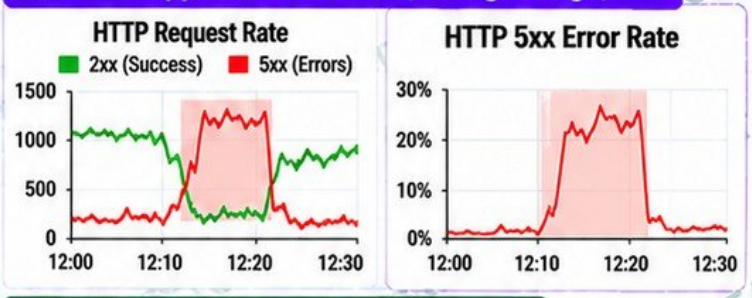
- How quickly does the alert fire?
- Does the alert identify customer impact?
- Is the alert based on sufficient request volume?
- Does the system avoid unnecessary notifications during harmless CPU spikes?
- Does the alert resolve after recovery?

3 Trigger the Outage (Generate 500 Errors)

```
# Option 1: Modify the API to return 500 errors
@app.route('/api/test-error')
def test_error():
    return "Internal Server Error", 500

# Option 2: Use a load script to generate 500s
for i in {1..1000}; do
    curl -s -o /dev/null -w "%{http_code}\n" \
    http://localhost:5000/api/test-error
done
```

4 Application Metrics (During Outage)



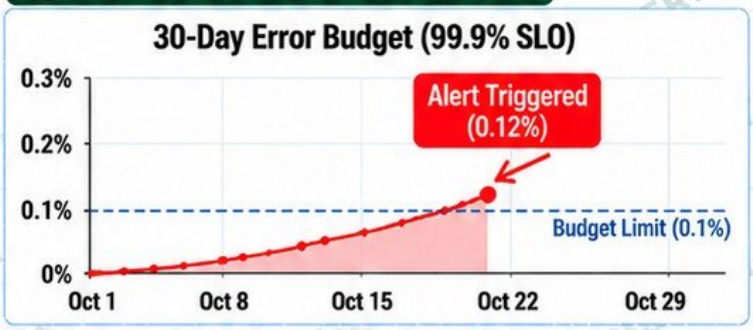
5 Alert Triggered (Alertmanager)

ALERT HighErrorBudgetBurn — Firing

```
Severity: page
Service: api
Summary: High error budget burn detected
Description: HTTP 5xx rate is 12.4% over 5m
(SLO 99.9%). Error budget will be exhausted
in 2.3 hours if this continues.
Started at: 2026-10-25 12:13:45
Runbook: https://github.com/veriqta/runbooks/api.md
```

View in Grafana | View in Prometheus | Runbook

6 Error Budget Consumption



7 Recovery (Alert Resolved)

ALERT HighErrorBudgetBurn — Resolved

```
Resolved at: 2026-10-25 12:28:10
Duration: 14m 25s
Final 5xx rate: 0.1%
Status: Service recovered
```

8 Verification Results

Check	Expected Result	Observed Result	Status
Alert fires quickly	< 5 minutes	2m 30s	Pass
Identifies customer impact	Shows 5xx rate & SLO	Correctly identified	Pass
Uses sufficient volume	Based on real traffic	Used 5m rate	Pass
Avoids CPU-only noise	No alert for CPU spike	No false alert	Pass
Resolves after recovery	Alert cleared	Resolved in 1m	Pass

9 GitHub Evidence

- Prometheus alert rules (YAML)
- Grafana dashboard definitions (JSON)
- Test script to generate 500 errors
- Alertmanager configuration
- Screenshots of alerts and dashboards
- Incident timeline and analysis

10 Engineering Insight

High CPU utilization is not automatically a customer incident. Rapid error-budget consumption may be.

11 Interview Defense

Q: Why might a 99.9% availability SLO require different alerting logic from a CPU threshold?

A:
1. SLOs measure customer impact, not infrastructure symptoms.
2. They require multi-window burn-rate alerts to avoid noise.
3. CPU thresholds can be noisy and may not affect user experience.
4. SLO alerts consider error budget, traffic volume, and real impact.

VERIQTA

Project 07: Build a Self-Service Developer Environment Platform

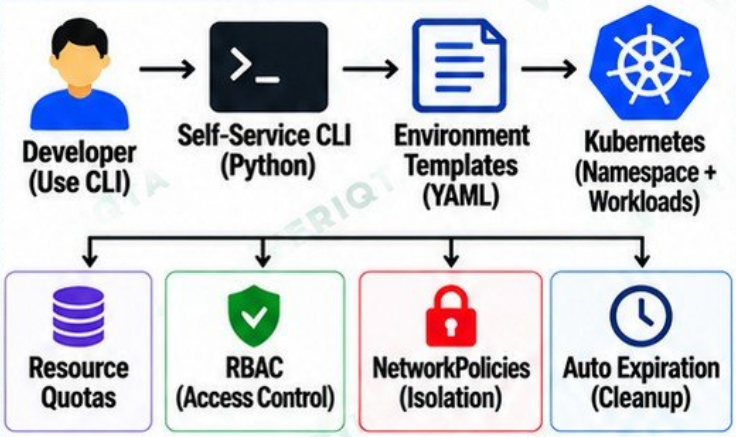
1 Scenario

Developers wait several days for an engineer to provision a testing environment.

2 Goal

- Build a lightweight internal platform that creates isolated development environments through a controlled interface.

3 Solution Architecture



4 Self-Service CLI (Python Example)

```python
#!/usr/bin/env python3
import argparse

def main():
    parser = argparse.ArgumentParser(
        description='Create a developer enviromment')
    parser.add_argument('--name', required=True,
        help='Environment name')
    parser.add_argument('--template', default='api',
        help='Environment template')
    parser.add_argument('--duration', default='7d',
        help='Auto expiration (e.g. 7d)')
    args = parser.parse_args()
    print(f"Creating environment: {args.name}")
    # Call Kubernetes API to create namespace and resources
    # Apply quotas, RBAC and network policies
if __name__ == "__main__":
    main()
```

5 Environment Templates (Example)

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev-api
  labels:
    owner: dev-team
    project: sample-api
    ttl: "7d"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: dev-api
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: api
        image: ghcr.io/veriqta/sample-api:latest
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits: "500m"
            memory: "S12Mi"
```

6 Provisioned Environment (Example)

```
$ kubectl get all -n dev-api

NAME                        READY   STATUS    RESTARTS   AGE
pod/api-6d8f6c8b7-abc12     1/1     Running   0          2m
pod/api-6d8f6c8b7-def34     1/1     Running   0          2m

NAME          TYPE        CLUSTER-IP     EXTERNAL-IP   AGE
service/api   ClusterIP   10.96.12.34    <none>        2m
```

Environment created successfully!

```
Name: dev-api
Template: api
Duration: 7 days
Owner: dev-team (user: alice)
Expires: 2026-11-01 12:00:00
```

7 Lifecycle Controls

- Standardized templates
- Resource quotas (CPU, memory, storage)
- RBAC (scoped permissions)
- NetworkPolicies (isolate environments)
- Automatic expiration (TTL labels)
- Environment ownership (user and team)
- Safe cleanup with ownership verification
- Record provisioning requests and results

8 Verification

- CLI creates a new namespace with workloads.
- Resource quotas are applied.
- RBAC restricts access to the namespace.
- NetworkPolicies are enforced.
- Environment expires automatically.
- Cleanup removes all resources safely.
- Provisioning requests and results are logged.

9 Engineering Insight

Platform engineering reduces repeated manual work by offering safe, repeatable capabilities rather than unrestricted infrastructure access.

10 Deliverable

- Working self-service CLI (Python).
- Kubernetes environment templates.
- Resource quotas, RBAC and NetworkPolicies.
- Automatic expiration and safe cleanup.
- Provisioning logs and audit records.
- Complete documentation and examples.

VERIQTA

Project 07: Prevent One Developer From Disrupting Everyone Else

1 Failure Challenge

A developer requests excessive resources and attempts to access another team's environment.

2 Investigation Questions

- Do quotas reject oversized requests?
- Does RBAC prevent unauthorized operations?
- Does NetworkPolicy enforce intended traffic restrictions?
- Are expired environments removed safely?
- Can the platform identify which developer owns each environment?

3 Resource Quotas (Prevent Excessive Resources)

```
# Developer tries to create a large pod
apiVersion: v1
kind: Pod
metadata:
  name: big-pod
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        cpu: "4"
        memory: "8Gi"
```

```
$ kubectl apply -f big-pod.yaml
Error from server (Forbidden):
pods "big-pod" is forbidden:
exceeded quota: dev-quota,
requested: limits.cpu=4,
limits.memory=8Gi,
used: limits.cpu=1,
limits.memory=2Gi,
limited: limits.cpu=2,
limits.memory=4Gi
```

Quota correctly rejected the request.

4 RBAC (Prevent Unauthorized Access)

```
$ kubectl auth can-i get pods -n team-b \
  --as=dev-user
no
```

RBAC denied access to another team's environment.

5 NetworkPolicy (Restrict Traffic)



6 Environment Expiration (Automatic Cleanup)

```
$ kubectl get ns
NAME        STATUS       AGE
dev-alice   Active       2d
dev-bob     Active       5d
dev-carol   Terminating  7d (expired)
```

```
$ kubectl describe ns dev-carol | grep -i owner
Owner:   carol
Expires: 2026-10-25T00:00:00Z
```

Expired environment is removed safely.

7 Environment Ownership (Identify Owner)

```
$ kubectl get ns -L owner,team,expires
NAME        STATUS  OWNER  TEAM       EXPIRES
dev-alice   Active  alice  payments   2026-10-30
dev-bob     Active  bob    platform   2026-10-31
dev-carol   Active  carol  marketing  2026-11-01
```

8 Verification Results

- Resource quotas reject oversized requests.
- RBAC prevents unauthorized operations.
- NetworkPolicy enforces traffic restrictions.
- Expired environments are removed safely.
- Platform identifies the owner of each environment.
- Audit logs show who created, accessed, and deleted environments.

9 GitHub Evidence

- Self-service CLI source code (Python).
- Kubernetes environment templates.
- ResourceQuota, RBAC and NetworkPolicy manifests.
- Automation for environment expiration and cleanup.
- Architecture diagram.
- Access-control test results and screenshots.
- README with setup, usage and examples.

10 Engineering Insight

Platform engineering reduces repeated manual work by offering safe, repeatable capabilities rather than unrestricted infrastructure access.

11 Interview Defense

Q: How would you balance developer self-service with infrastructure governance?

A: Provide self-service through templates and automation, enforce guardrails with quotas, RBAC and NetworkPolicy, use clear ownership and expiration, maintain audit logs, and continuously monitor for abuse or misconfiguration.

VERIQTA

Project 08: Build a Disaster Recovery System That Proves Backups Work

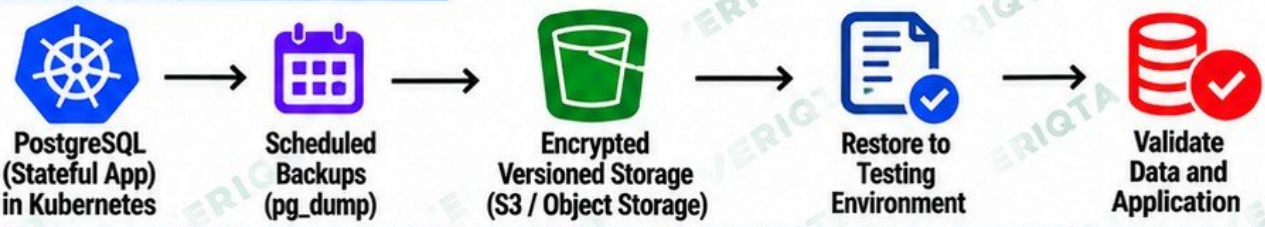
1 Scenario

A team performs daily backups but discovers during an outage that the backup cannot restore the application.

2 Goal

- Build an automated backup and restoration validation system.
- Prove that backups can be successfully restored.
- Measure recovery time and data recovery point.

3 Architecture Overview



4 Database Backup (PostgreSQL)

```
# Full backup using pg_dump
DB_NAME="appdb"
BACKUP_FILE="appdb_$(date +-%F).sql"
pg_dump -h $DB_HOST -U $DB_USER -d $DB_NAME \
  -Fc -f $BACKUP_FILE

# Encrypt backup
gpg --symmetric --cipher-algo AES256 \
  --output ${BACKUP_FILE}.gpg $BACKUP_FILE

# Upload to object storage (example: AWS S3)
aws s3 cp ${BACKUP_FILE}.gpg \
  s3://veriqta-backups/postgresql/ \
  --storage-class STANDARD_IA --sse AES256
```

5 Storage and Security

- Store backups in protected, versioned object storage (S3 / compatible).
- Encrypt backup data at rest and in transit.
- Restrict restoration permissions (least privilege IAM / RBAC).
- Enable versioning and lifecycle policies (retention).

6 Restore to Testing Environment

```
# Download and decrypt backup
aws s3 cp  s3://veriqta-backups/
postgresql/appdb_2026-10-25.sql.gpg .
gpg --output appdb_restore.sql \
   --decrypt appdb_2026-10-25.sql.gpg

# Restore to test environment
kubectl exec -i postgres-test -- \
  dropdb appdb_test
kubectl exec -i postgres-test -- \
  createdb appdb_test
kubectl exec -i postgres-test -- \
  pg_restore -d appdb_test \
  appdb_restore.sql
```

7 Validation Checks

- Database restore completes successfully.
- Application connects and runs queries.
- Data consistency is verified (row counts, key records).
- Recovery time (RTO) is recorded.
- Data recovery point (RPO) is measured.
- Automated test runs on a schedule.

8 Example Validation (SQL)

```
# Verify tables and row counts
psql -h postgres-test -U app -d appdb_test -c
"/dt"

psql -h postgres-test -U app -d appdb_test -c
"SELECT COUNT(*) FROM users;"
```

Table	Row Count
users	1250
orders	5430
products	980
audit_logs	2310

9 Recovery Metrics

Metric	Result
Backup Frequency	Daily
Last Backup Time	2026-10-25 01:00
Recovery Time (RTO)	12 minutes
Data Recovery Point (RPO)	< 24 hours
Data Consistency	Verified ✓
Application Health	Running ✓

10 Engineering Insight

A successful backup job does not prove recoverability.

11 Deliverable

- Automated backup and restore system.
- PostgreSQL deployment and Kubernetes manifests.
- Scheduled backup scripts.
- Encrypted, versioned storage configuration.
- Restore validation tests.
- Recovery time and data recovery point report.

12 GitHub Evidence

- Kubernetes manifests (PostgreSQL).
- Backup and restore scripts.
- Storage and encryption configuration.
- Test environment setup.
- Validation scripts and query results.
- Documentation with RTO and RPO.
- README with setup and run instructions.

13 Interview Defense

Q: Why is a successful backup job not enough?

A: A successful job only confirms that a backup file was created. It does not prove that the backup can be restored, that the data is consistent, or that the recovery process works within the required time and access controls.

VERIQTA

Project 08: Restore a Database After a Simulated Failure

1 Failure Challenge

Destroy a disposable test database and restore it from the latest verified backup.

2 Investigation Questions

- Is the backup complete and readable?
- Can the application connect after restoration?
- How much data was lost relative to the backup point?
- Did recovery finish within the recovery time objective?
- What happens if the latest backup is corrupt?

3 Failure and Recovery Flow



4 Simulate Database Failure

```
# Identify the test database pod
kubectl get pods -l app=appdb-test

# Delete the database pod (simulates failure)
kubectl delete pod appdb-test-0

# Verify the application is down
kubectl get pods -l app=web
```

Application cannot connect to the database.

5 Restore from Backup

```
# Download latest backup from S3
aws s3 cp s3://veriqta-backups/postgres/
latest/appdb_2026-10-25.sql.gz .

# Restore to a new database
gunzip appdb_2026-10-25.sql.gz
psql -h postgres-new -U postgres \
-d appdb_test -f appdb_2026-10-25.sql

# Verify restoration
psql -h postgres-new -U postgres \
-d appdb_test -c "SELECT COUNT(*) FROM users;"
```

6 Application Validation

- Application connects to restored database.
- Key tables and records exist.
- Application functionality works as expected.
- Data consistency checks pass.
- No errors in application logs.

Application is running with restored data.

7 Recovery Metrics (Example)

Metric	Target	Actual Result	Status
Backup Point (RPO)	≤ 24 hours	1 hour ago	Pass
Recovery Time (RTO)	≤ 30 minutes	12 minutes	Pass
Data Loss	0 rows	0 rows	Pass
Application Health	Running	Running	Pass
Data Consistency	Verified	Verified	Pass

8 Handle a Corrupt Backup (Fallback)

```
# Attempt to restore corrupt backup
gunzip corrupt_backup.sql.gz
psql -h postgres-new -U postgres -d appdb_test \
-f corrupt_backup.sql
ERROR:  invalid input syntax (corrupt file)

# Use previous healthy backup
aws s3 ls s3://veriqta-backups/postgres/
# Restore from previous good backup
psql -h postgres-new -U postgres -d appdb_test \
-f appdb_2026-10-24.sql
```

Successfully restored using previous backup.

9 Verification Results

- Database successfully restored from backup.
- Application validated and accessible.
- Measured recovery time within target (RTO).
- No data loss beyond recovery point (RPO).
- Fallback backup tested successfully.
- Recovery report generated with evidence.

10 GitHub Evidence

- Backup scripts (PostgreSQL).
- Restore workflow scripts.
- Validation test scripts.
- Recovery test results and logs.
- Disaster recovery runbook.
- Screenshots and metrics.
- README with setup and instructions.

11 Engineering Insight

A successful backup job does not prove recoverability. Regular restoration testing is essential.

12 Interview Defense

Q: Why is a backup retention policy not the same thing as a disaster recovery strategy?

A: A retention policy defines how long backups are kept. A disaster recovery strategy includes backup, restoration testing, RTO/RPO targets, infrastructure, procedures, and validation to ensure actual recovery.

VERIQTA

Project 09: Build a Container Supply-Chain Trust Gateway

1 Scenario

A container passes vulnerability scanning but originates from an untrusted build process.

2 Goal

- Build a release verification system that checks whether container images satisfy defined security requirements.
- Enforce artifact authenticity and security policies before deployment.

3 Release Verification Pipeline (Overview)



1. Build and Publish Image (versioned)
2. Generate SBOM (Syft)
3. Scan Image and Dependencies (Trivy)
4. Sign Image and Verify (Cosign)
5. Record Build Provenance (SLSA/Attestations)
6. Enforce with Kubernetes Admission Policy (Reject unsigned)

4 Build and Publish Image

```
# Build and tag a versioned image
docker build -t veriqta/app:1.0.0 .
docker push ghcr.io/veriqta/app:1.0.0

# Get immutable digest
docker inspect --format='{{index
.RepoDigests 0}}' veriqta/app:1.0.0
```

Image published with version and immutable digest.

5 Generate SBOM (Syft)

```
# Generate SBOM in SPDX format
syft ghcr.io/veriqta/app:1.0.0 \\
  -o spdx-json > sbom.json
```

SBOM created successfullly.

```
{
  "name": "veriqta/app",
  "version": "1.0.0",
  "sbomFormat": "SPDX",
  "packages": [...]
}
```

6 Scan for Vulnerabilities (Trivy)

```
# Scan image for vulnerabilities
trivy image ghcr.io/veriqta/app:1.0.0
```

SEVERITY	COUNT	ACTION
CRITICAL	0	Pass
HIGH	2	Fix / Review
MEDIUM	5	Document
LOW	12	Informational

7 Sign and Verify Image (Cosign)

```
# Sign the image
cosign sign --key cosign.key \
    ghcr.io/veriqta/app:1.0.0
# Verify signature
cosign verify --key cosign.pub \
    ghcr.io/veriqta/app:1.0.0
```

Image signature verified.

8 Build Provenance (SLSA)

```
# Generate provenance attestation
cosign attest --predicate \
    provenance.json \
    --type slsaprovenance
    ghcr.io/veriqta/app:1.0.0
```

Build provenance recorded and linked to image digest.

9 Kubernetes Admission Policy

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: verify-signed-images
rules:
- apiGroups: [""]
  resources: ["pods"]
  operations: ["CREATE"]
```

Reject unsigned or unapproved images at deploy time.

10 Security Findings and Exceptions

FINDING TYPE	ACTION	EXAMPLE
Critical Vulnerability	Block Deployment	Remote code execution
High Vulnerability	Block or Fix	Privilege escalation
Medium Vulnerability	Document Exception	Non-exploitable in runtime
Low Vulnerability	Informational	Update in next release
Unsigned Image	Block Deployment	Missing Cosign signature
Unknown Builder	Block Deployment	Untrusted build process

11 Engineering Insight

Vulnerability scanning and artifact authenticity solve different security problems. An image can have no known vulnerabilities yet still be untrusted if its source, build process, or signature cannot be verified.

12 Deliverable

- Container release pipeline.
- SBOM generation (Syft).
- Vulnerability scanning (Trivy).
- Image signing and verification (Cosign).
- Build provenance and immutable digests.
- Kubernetes admission policy.
- Exception management process.
- Complete documentation and examples.

VERIQTA

Project 09: Block a Valid Image From an Untrusted Source

1 Failure Challenge

Attempt to deploy an unsigned image containing no known critical vulnerabilities.

2 Investigation Questions

- Does the image pass vulnerability scanning?
- Is its signature missing or invalid?
- Does the admission policy reject it?
- Can a signed, approved image deploy successfully?
- Are policy exceptions controlled and auditable?

3 Test Flow (Unsigned vs Signed)



4 Vulnerability Scan Result (Trivy)

```
$ trivy image ghcr.io/veriqta/app:1.0.0
Total: 25 (UNKNOWN: 5, LOW: 18, MEDIUM: 2,
HIGH: 0, CRITICAL: 0)
```

SEVERITY	COUNT	STATUS
CRITICAL	0	Pass
HIGH	0	Pass
MEDIUM	2	Document
LOW	18	Document
UNKNOWN	5	Informational

Image has no critical vulnerabilities.

5 Check Image Signature (Cosign)

```
$ cosign verify ghcr.io/veriqta/app:1.0.0

Error: no signatures found
no matching signatures:
  none found for object
  ghcr.io/veriqta/app:1.0.0
```

Image is unsigned or signature is invalid.

6 Admission Policy (Reject Unsigned)

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: verify-signed-images
rules:
- apiGroups: [""]
  resources: ["pods"]
  operations: ["CREATE"]
  failurePolicy: Fail
  matchPolicy: Equivalent
```

Reject containers that are unsigned or not from approved sources.

7 Deployment Attempt (Unsigned Image)

```
$ kubectl apply -f deployment-unsigned.yaml

Error from server (Forbidden):
admission webhook "verify-signed-images"
denied the request: image is not signed
or from an approved source
```

Deployment rejected by admission policy.

8 Deploy Signed and Approved Image

```
$ cosign sign --key cosign.key \
  ghcr.io/veriqta/app:1.0.0
$ kubectl apply -f deployment-signed.yaml

deployment.apps/app created
```

Signed and approved image deploys successfully.

9 Policy Exceptions (Controlled)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: image-exceptions
  namespace: security
data:
  allowed-images: |
    ghcr.io/veriqta/base-image:1.0.0
    registry.example.com/approved/*
```

Exceptions are explicitly listed, reviewed and auditable.

10 Verification Results

- Unsigned image is rejected.
- Vulnerability scan shows no critical issues.
- Signed image deploys successfully.
- Admission policy audit logs show the event.
- Policy exceptions are controlled and auditable.

11 GitHub Evidence

- CI/CD workflows (build, scan, sign, deploy).
- Image attestations and signatures.
- Kubernetes admission policy manifests.
- Security verification test results.
- Audit logs and screenshots.
- README with setup and run instructions.

12 Engineering Insight

Vulnerability scanning and artifact authenticity solve different security problems. An image with zero reported vulnerabilities can still be untrusted and must be rejected.

13 Interview Defense

Q: Why should a container with zero reported vulnerabilities still be rejected from production?

A: Because vulnerability scanning only checks for known flaws. An unsigned image could come from an untrusted source, be tampered with, or not follow the organization's build and supply-chain security process.

VERIQTA

Project 10: Build an Incident Replay and Root-Cause Evidence Engine

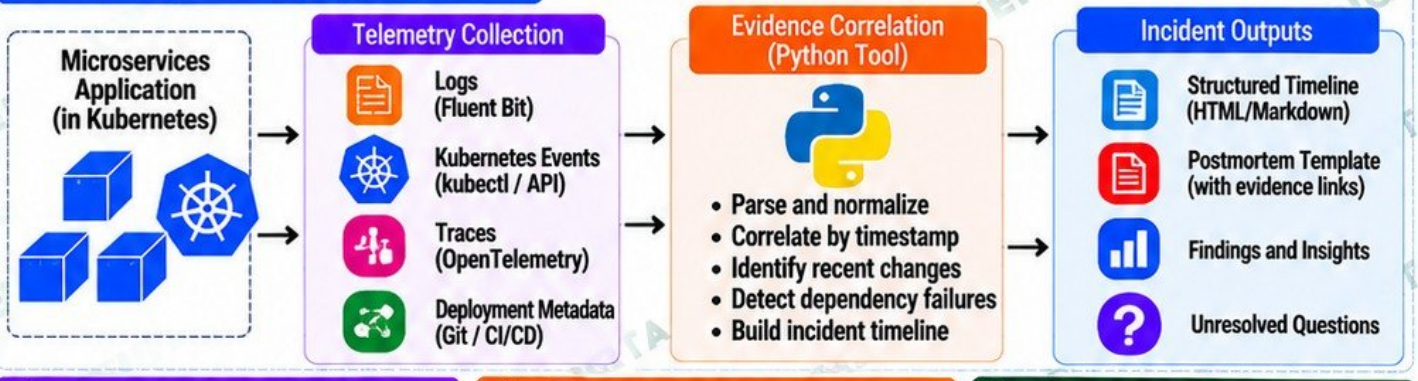
1 Scenario

A production outage ends, but engineers cannot confidently explain what caused it because logs, events, and deployment history are scattered across multiple tools.

2 Goals

- Build a lightweight incident evidence collector that reconstructs an incident timeline.
- Correlate logs, events, traces, and deployments by timestamps and service identity.
- Identify recent changes and dependency failures.
- Generate a postmortem template with evidence links and unresolved questions.

3 Architecture Overview



4 Collect Application Logs

```
# Get logs from a microservice
kubectl logs -n app deployment/api \
  --since=2h > logs/api.log

# Use Fluent Bit (example config)
[SERVICE]
  Flush        5
  Log_Level    info
  Output       es

[INPUT]
  Name         tail
  Path         /var/log/containers/*.log
```

5 Collect Kubernetes Events

```
# Get recent events
kubectl get events -n app \
  --sort-by=.lastTimestamp \
  > events/cluster-events.log

# Watch specific resource events
kubectl get events -n app --field-selector \
  involvedObject.kind=Pod \
  --sort-by=.lastTimestamp
```

TIME	TYPE	REASON	MESSAGE
10:14	Warning	BackOff	Back-off restarting pod
10:16	Normal	Pulled	Successfully pulled image
10:18	Warning	Failed	Readiness probe failed

6 Collect Traces (OpenTelemetry)

```
# Instrument application (example)
from opentelemetry import trace
from opentelemetry.sdk.trace import
  TracerProvider
from opentelemetry.exporter.otlp.proto.grpc
  import OTLPSpanExporter

provider = TracerProvider()
exporter = OTLPSpanExporter(
  endpoint="http://otel-collector:4317"
)
provider.add_span_processor(
  BatchSpanProcessor(exporter)
)
```

7 Incident Correlation Tool (Python)

```
# Run the evidence correlation tool
python incident_replay.py \
  --logs logs/ \
  --events events/
  --traces traces/
  --deployments deployments/
  --output timeline.html
```

8 Example Incident Timeline

TIME	SOURCE	SERVICE	EVENT
10:14	K8s Event	api	Pod restarted (BackOff)
10:15	Deployment	web	New image deployed
10:16	Logs	api	Database connection error
10:17	Trace	api	Increased latency to db
10:18	K8s Event	db	Readiness probe failed
10:20	Logs	db	Out of memory
10:22	Deployment	api	Rollback to previous version
10:25	Trace	api	Latency returned to normal

9 Postmortem Template

- Incident summary
- Impact and user experience
- Timeline with evidence links
- Root cause analysis
- Contributing factors
- What went well / what didn't
- Action items
- Unresolved questions

10 Verification Results

- Logs, events, traces and deployments collected successfully.
- Evidence correlated by timestamps.
- Structured incident timeline generated.
- Postmortem template completed.
- Investigation toolkit is reproducible.

11 Engineering Insight

Temporal correlation helps narrow an investigation, but it does not establish causation by itself. Multiple evidence sources and deeper analysis are required.

12 Deliverable

- Reproducible incident investigation toolkit (Python).
- Sample microservices application.
- Collection scripts and configuration.
- Generated incident timeline.
- Postmortem template with evidence links.
- README with setup and usage instructions.

13 Interview Defense

Q: Why is temporal correlation important in incident investigations, and why does it not prove causation?

A: Temporal correlation helps narrow the scope by showing what changed and when. It does not prove causation because multiple factors can occur at the same time, and further analysis and testing are needed.

VERIQTA

Project 10: Reconstruct an Outage Without Guessing the Root Cause

1 Failure Challenge

Deploy a version that introduces database connection pool exhaustion during increased request traffic.

2 Investigation Questions

- Which symptom appeared first?
- Did the failure begin after a deployment?
- Were requests waiting for database connections?
- Did retries amplify the problem?
- Which traces, logs, and metrics support the leading hypothesis?
- Can the failure be reproduced and eliminated through a controlled change?

3 Incident Flow (Overview)



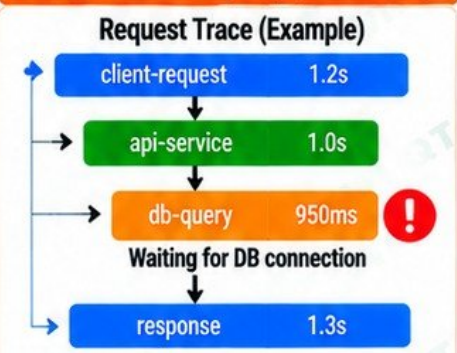
4 Identify the Failure (Logs)

```
# Application logs (example)
2026-10-25 10:14:21 ERROR
HikariPool-1 - Connection is not available,
request timed out after 30000ms.

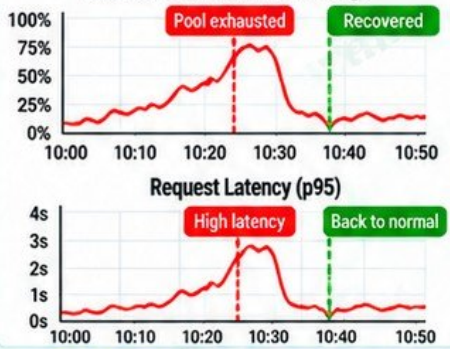
2026-10-25 10:14:22 WARN
Retry attempt 3/3 for query

2026-10-25 10:14:23 ERROR
SQLTransientConnectionException:
Connection pool exhausted.
```

5 Trace Analysis (OpenTelemetry)



6 Metrics (Before vs After)



7 Reproduce and Fix

```
# Deploy the problematic version
kubectl apply -f deployment-bad.yaml

# Generate load
hey -n 5000 -c 100 http://api.local

# Apply fix (increase pool size)
kubectl apply -f deployment-fixed.yaml
```

8 Incident Timeline (Example)

Time	Event	Evidence Source
10:12	New version deployed	Kubernetes events
10:14	DB connection wait time increases	Traces / Metrics
10:15	Application errors (5xx)	Logs / Metrics
10:16	Retries increase traffic	Logs / Traces
10:20	Root cause identified	Correlated evidence
10:25	Fix deployed (increase pool size)	Deployment events
10:27	System recovers	Metrics / Traces

9 Postmortem Template

- Incident summary
- Impact and duration
- Timeline with evidence links
- Root cause analysis
- Contributing factors
- What went well / what didn't
- Action items
- Unresolved questions

10 Verification Results

- Failure reproduced successfully.
- Incident timeline generated.
- Evidence from logs, metrics and traces.
- Root cause confirmed.
- Fix applied and verified.
- Measurable recovery observed.

11 GitHub Evidence

- Incident timeline generator (Python).
- OpenTelemetry configuration.
- Sample application and manifests.
- Sanitized traces, logs and metrics.
- Postmortem document.
- Verification report with before/after results.
- README with setup and run instructions.

12 Engineering Insight

Temporal correlation helps narrow an investigation, but it does not establish causation by itself. Multiple sources of evidence are required to reach a reliable conclusion.

13 Interview Defense

Q: How do you distinguish a plausible root-cause theory from one supported by sufficient production evidence?

A: A plausible theory is a possible explanation, but it is not proven. A reliable root cause is supported by consistent evidence from multiple sources, can be reproduced, and is eliminated through a controlled change.